

Introduction to Algorithms

3rd Edition

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, & Clifford Stein

Graham Strickland

August 30, 2026

Contents

1	The Role of Algorithms in Computing	2
1.1	Algorithms	2
1.2	Algorithms as a technology	3
2	Getting Started	4
2.1	Insertion sort	4
2.2	Analyzing algorithms	7
2.3	Designing algorithms	8
3	Growth of Functions	17
3.1	Asymptotic notation	17
3.2	Standard notations and common functions	19
4	Divide-and-Conquer	21
4.1	The maximum-subarray problem	21
4.2	Strassen's algorithm for matrix multiplication	23
A	Summations	26
A.1	Summation formulas and properties	26

1 The Role of Algorithms in Computing

1.1 Algorithms

1.1-1

Suppose we need to find the smallest number of buildings in a suburb of a city which need security huts for surveillance of vehicles entering the suburb. We can model this problem by trying to find the convex hull, where the buildings are the n points in the plane and the huts are placed at its vertices.

1.1-2

Space required for data storage, since memory is finite in any computing system executing an algorithm.

1.1-3

Linked lists have the following strengths:

- They are easy to insert items into.
- They are easy to remove items from.
- They do not require a large block of contiguous memory to be allocated ahead of insertion.

They have the following limitations:

- Traversing the linked list is slow.
- Accessing an element is not easy and requires indirect methods.
- Deallocating the memory used is slow.

1.1-4

The traveling-salesman problem can be interpreted as a series of many different shortest-path problems which are combined into one. However, the shortest-path problem does not require that whoever traverses the path return to where they started. It can also be solved efficiently, whereas the traveling-salesman problem cannot.

1.1-5

If we are trying to find the optimal layout of train routes so that trains do not cross tracks at the same time, then only the best solution will do. On the other hand, if we are trying to minimize the cost of procuring parts for the train, then an approximation to the best solution may be good enough.

1.2 Algorithms as a technology

1.2-1

A Google search uses algorithmic content to find matches to a search query in the shortest time possible. The algorithms used must find the shortest path between various networks of web pages as well as parse through large amounts of text data in the shortest time possible.

1.2-2

Insertion sort beats merge sort exactly when

$$\begin{aligned} 8n^2 &< 64n \lg n \\ \Leftrightarrow n &< 8 \lg n, \end{aligned}$$

dividing both sides by $8n$, which is positive for all $n \geq 1$.

The function $8 \lg n - n$ is positive at $n = 2$ and, since $\lg n$ grows more slowly than n , changes sign exactly once. We therefore need the largest integer n satisfying $n < 8 \lg n$:

$$\begin{aligned} n = 43 : \quad 8 \lg 43 &\approx 43.42 > 43, \\ n = 44 : \quad 8 \lg 44 &\approx 43.68 < 44. \end{aligned}$$

Thus insertion sort beats merge sort for $2 \leq n \leq 43$, and merge sort is at least as fast for every $n \geq 44$.

1.2-3

We are looking for the interval where $100n^2 < 2^n$, thus for $n = 15$, n^2 runs faster than 2^n .

Problems

1-1

From the program in `cpp/compruntime/main.cpp`, we have Table 1.

In generating the above results, we have made use of the following two C++ functions in `cpp/algorithms/big_oh.cpp`:

```
double inverse_nlogn(double x) {
    std::size_t max_iters = 10;
    double a_0 = x / std::log2(x), a_1 = 0.0;

    for (std::size_t i = 0; i < max_iters; ++i) {
        a_1 = a_0 - ((a_0 * std::log2(a_0) - x) /
                     ((1.0 / std::log(2.0)) + std::log2(a_0)));
        if (std::abs((a_1 * std::log2(a_1)) - (a_0 * std::log2(a_0))) < 1)
            return a_1;
        else
            continue;
    }
}
```

$f(n)$	1 second	1 minute	1 hour	1 day	1 month	1 year	1 century
$\lg n$	inf	inf	inf	inf	inf	inf	inf
$\sqrt{n}$	1e+12	3.6e+15	1.296e+19	7.46496e+21	6.71846e+24	9.94519e+26	9.94519e+30
n	1e+06	6e+07	3.6e+09	8.64e+10	2.592e+12	3.1536e+13	3.1536e+15
$n \lg n$	62746	2.80142e+06	1.33378e+08	2.75515e+09	7.18709e+10	7.97634e+11	6.8611e+13
n^2	1000	7745	60000	293938	1.60997e+06	5.61569e+06	5.61569e+07
n^3	100	391	1532	4420	13736	31593	146645
2^n	19	25	31	36	41	44	51
$n!$	9	11	12	13	15	16	17

Table 1: Comparison of running times for the largest problem size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds

```

    a_0 = a_1;
}

```

```

return a_1;
}

```

and:

```

double inverse_factorial(double x) {
    double n = 0, fact = 1;

    while (fact * (n + 1) <= x) {
        n++;
        fact *= n;
    }

    return n;
}

```

`inverse_nlogn` makes use of Newton's method to calculate an approximate value $m(x)$ s.t.

$$n \lg n = x \Rightarrow m(x) \approx n,$$

while `inverse_factorial` generates an approximate value $m(x)$ s.t.

$$x = n! \Rightarrow m(x) = \lfloor n \rfloor.$$

2 Getting Started

2.1 Insertion sort

2.1-1

In Figure 1, we illustrate the operation of INSERTION-SORT on the array $A = \langle 31, 41, 59, 26, 41, 58 \rangle$.

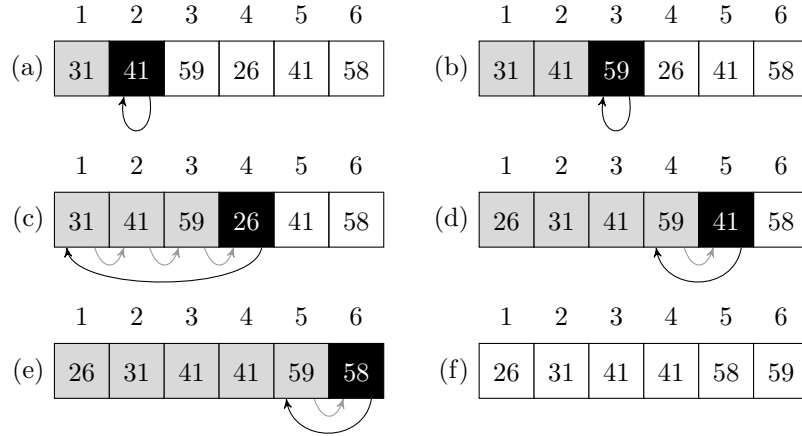


Figure 1: Illustration of INSERTION-SORT.

2.1-2

INSERTION-SORT(A)

```

1: for  $j = 2$  to  $A.length$  do
2:    $key = A[j]$ 
3:   // Insert  $A[j]$  to the sorted
   sequence  $A[1..j-1]$ 
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] < key$  do
6:      $A[i+1] = A[i]$ 
7:     if  $i > 0$  then
8:        $i = i - 1$ 
9:     else
10:       $i = 0$ 
11:     $A[i+1] = key$ 

```

2.1-3

LINEAR-SEARCH(A, ν)

```

1:  $j = 1$ 
2: while  $j \leq A.length$  do
3:   if  $A[j] == \nu$  then
4:     return  $j$ 
5:   else
6:      $j = j + 1$ 
7: return NIL

```

Initialisation: With the loop invariant being that $A[j]$ refers to an element of the sequence $A = \langle a_1, a_2, \dots, a_n \rangle$ and each element in $A[1..j-1]$ has

been checked for equality, we have the loop invariant valid, since $j = 1$ and $A[1]$ is in A for $n \geq 1$, at the start of the **while** loop 1 – 6.

Maintenance: Either the **if** statement in line 3 returns $A[j]$ or the **else** statement in line 5 increments j by 1, so that after each pass of the **while** loop, we have checked whether or not $A[j] = \nu$, and the condition of the **while** loop ensures that $A[j]$ is in A .

Termination: If the **while** loop terminates, then either the **if** condition in line 3 is true, so that $A[j] = \nu$ or $j > A.length$, at which point we return the special value NIL. The loop condition must be $j \leq A.length$ rather than $j \neq A.length$, since the latter halts before $A[n]$ is ever compared with ν .

Thus the algorithm is correct, since the loop invariant is initialised and maintained throughout, and the algorithm terminates with the correct output; if a value is returned, ν is in A , and occurs at $A[j]$ for return value j , otherwise the special value NIL was returned, and ν is not in A .

2.1-4

We have the following addition problem:

Input: Two n -element arrays A and B containing binary digits.

Output: $(n + 1)$ -element array C containing the sum of A and B .

We have the following algorithm as a solution:

```

BINARY-ADDITION( $A, B, n$ )
1:  $carry = 0$ 
2: for  $i = 1$  to  $n$  do
3:   if  $A[i] = 1$  and  $B[i] = 1$  then
4:     if  $carry == 0$  then
5:        $C[i] = 0$ 
6:        $carry = 1$ 
7:     else
8:        $C[i] = 1$ 
9:        $carry = 1$ 
10:  else if  $A[i] == 1$  and  $B[i] == 0$  or
       $A[i] == 0$  and  $B[i] == 1$  then
11:    if  $carry == 0$  then
12:       $C[i] = 1$ 
13:    else
14:       $C[i] = 0$ 
15:       $carry = 1$ 
16:  else
17:     $C[i] = carry$ 
18:     $carry = 0$ 
19:  $C[n + 1] = carry$ 
20: return  $C$ 

```

2.2 Analyzing algorithms

2.2-1

$$\frac{n^3}{1000} - 1000n^2 - 100n + 3 = \Theta(n^3).$$

2.2-2

SELECTION-SORT(A)

```
1: for  $i = 1$  to  $A.length - 1$  do
2:    $smallest = i$ 
3:   for  $j = i + 1$  to  $A.length$  do
4:     if  $A[j] < A[smallest]$  then
5:        $smallest = j$ 
6:   exchange  $A[i]$  with  $A[smallest]$ 
```

The **for** loop from lines 1-6 maintains the loop invariant that $A[1..i-1]$ contains sorted elements in ascending order.

Line 6 must exchange the two elements rather than merely assign $A[i]$ to $A[smallest]$, which would overwrite the smallest element and lose the value it replaced.

The inner **for** loop of lines 3-5 may begin at $j = i + 1$, since $smallest$ is initialised to i and the comparison of $A[i]$ with itself can never succeed.

The algorithm only needs to run for the first $n - 1$ elements since the n th element will already be the largest element in the array after each smaller element in $A[1..n]$ has been sorted.

Since there is no **while** loop and each search for the smallest item in the subarray $A[i+1..n]$ must occur, the best and worst-case running times are the same, i.e., $\Theta(n^2)$.

2.2-3

Assuming the element being searched for is equally likely to be at any position in the array, we would on average search through

$$\frac{1 + 2 + \dots + n}{n} = \frac{1}{n} \sum_{k=1}^n k = \frac{n+1}{2}$$

elements for an input sequence of length n . In the worst case we would search through all n elements.

Thus the average- and worst-case running times are $\Theta(n)$, since

$$\frac{n+1}{2} = \frac{n}{2} + \frac{1}{2},$$

and the dominant term is $\frac{1}{2}n$, which is equivalent to $\Theta(n)$. Obviously, looping through n items sequentially is $\Theta(n)$, so the average- and worst-case running times are in an equivalent Θ -class.

2.2-4

By only inputting elements that are equivalent or close to the expected output.

2.3 Designing algorithms

2.3-1

In Figure 2, we illustrate the operation of MERGE-SORT on the array $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

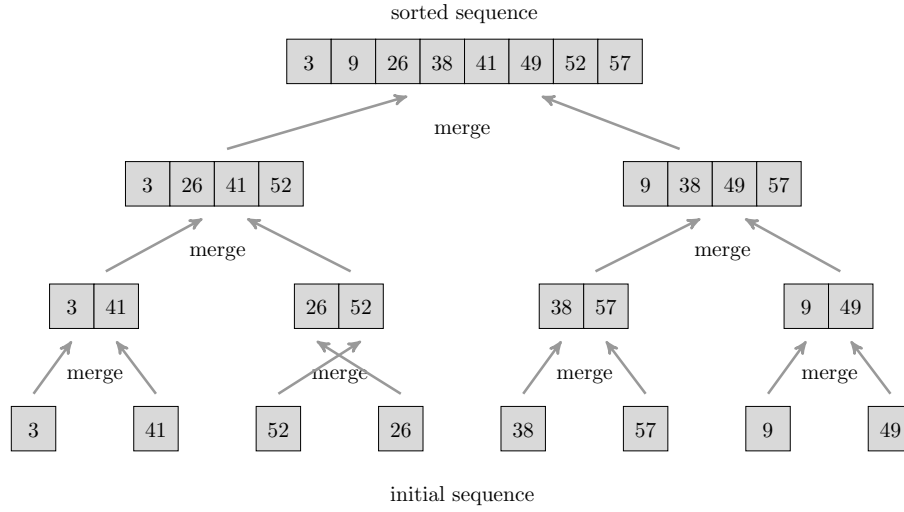


Figure 2: Illustration of MERGE-SORT.

2.3-2

```
MERGE( $A, p, q, r$ )
1:  $n_1 = q - p + 1$ 
2:  $n_2 = r - q$ 
3: let  $L[1..n_1]$  and  $R[1..n_2]$  be new arrays
4: for  $i = 1$  to  $n_1$  do
5:    $L[i] = A[p + i - 1]$ 
6: for  $j = 1$  to  $n_2$  do
7:    $R[j] = A[q + j]$ 
8:  $i = 1$ 
9:  $j = 1$ 
10: for  $k = p$  to  $r$  do
11:   if  $i > n_1$  then
12:      $A[k] = R[j]$ 
```



```

13:      $j = j + 1$ 
14:   else if  $j > n_2$  then
15:      $A[k] = L[i]$ 
16:      $i = i + 1$ 
17:   else if  $L[i] \leq R[j]$  then
18:      $A[k] = L[i]$ 
19:      $i = i + 1$ 
20:   else
21:      $A[k] = R[j]$ 
22:      $j = j + 1$ 

```

Without the sentinels, L and R hold exactly n_1 and n_2 elements, and the **for** loop of lines 10-21 must first test whether either array has been exhausted before comparing $L[i]$ with $R[j]$. Once one array is spent, the remaining elements of the other are copied across unconditionally.

2.3-3

Proof. We have the base case for $n = 2^1$ given by

$$T(n) = T(2) = 2 = 2 \log_2 2^1 = 2 \cdot 1.$$

Now, we assume as our induction step that if $n = 2^k$ for $k \geq 1$,

$$T(n) = 2T(n/2) + n = n \lg n.$$

Then, suppose that $n = 2^{k+1}$, so that we have

$$\begin{aligned}
T(n) &= 2T(2^{k+1}/2) + 2^{k+1} \\
&= 2T(2^k) + 2^{k+1} \\
&= 2(2^k \lg 2^k) + 2^{k+1} && \text{(by our induction assumption)} \\
&= 2^{k+1} \lg 2^k + 2^{k+1} \lg 2 && \text{(since } \log_2 2 = 1) \\
&= 2^{k+1} \lg 2^{k+1} && \text{(since } \log_2 a + \log_2 b = \log_2 ab)
\end{aligned}$$

Therefore, by induction on $n = 2^k$, $T(n) = n \lg n$. $\square$

2.3-4

We let $T(n)$ be the worst-case running time of sorting an array of n elements. For $n = 1$, we have $T(n) = \Theta(1)$, since a one-element array is already sorted and no work is required.

If $n > 1$, the recursion sorts $A[1..n-1]$ and then inserts $A[n]$ into that sorted subarray. This yields a *single* subproblem of size $n-1$, not several, so the recursive call costs $T(n-1)$. The insertion itself must shift every element of $A[1..n-1]$ that exceeds $A[n]$ one position to the right, which in the worst case is all $n-1$ of them, and so costs $\Theta(n)$. Therefore

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ T(n-1) + \Theta(n) & \text{otherwise} \end{cases}.$$

Unrolling the recurrence gives

$$T(n) = \sum_{k=1}^n \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2),$$

the same worst-case running time as the iterative INSERTION-SORT.

2.3-5

BINARY-SEARCH(A, ν)

```

1:  $low = 1$ 
2:  $high = A.length$ 
3: while  $low \leq high$  do
4:    $mid = \lfloor \frac{high+low}{2} \rfloor$ 
5:   if  $A[mid] == \nu$  then
6:     return  $mid$ 
7:   else if  $A[mid] > \nu$  then
8:      $high = mid - 1$ 
9:   else
10:     $low = mid + 1$ 
11: return  $-1$ 

```

Suppose we have an array A of length n . Then, in the worst case, we loop through the **while** loop of BINARY-SEARCH (lines 3-9) until $low > high$ and return -1 , so that ν is not in A . The comparison of line 3 must be $low \leq high$ rather than $low < high$, since otherwise a one-element range is never examined and the search misses any ν that lies in it. If we continually divide n by 2, and n is a power of 2, then we will execute

$$\lfloor \frac{high - low}{2} \rfloor$$

exactly $\lg n$ times. If n is not a power of 2, then the same operation executes $\lfloor \lg n \rfloor$ times. In any case, we have a worst-case running time of $\Theta(\lg n)$ for BINARY-SEARCH.

2.3-6

Since the loop invariant of INSERTION-SORT guarantees that we have $A[1..j-1]$ sorted, we can indeed use the BINARY-SEARCH algorithm of 2.3-5 to locate the position at which $A[j]$ belongs, reducing the number of *comparisons* for that insertion from $\Theta(j)$ to $\Theta(\lg j)$.

This does not, however, improve the worst-case running time to $\Theta(n \lg n)$. The **while** loop of lines 5-7 does not only compare elements; it also shifts each element of $A[1..j-1]$ that exceeds $A[j]$ one position to the right. Binary search locates the insertion point but performs none of that movement, and in the worst case, when A is in decreasing order, all $j-1$ elements must still be

moved. Summing over every iteration of the outer loop gives

$$\sum_{j=2}^n \Theta(j) = \Theta(n^2)$$

element movements regardless of how the insertion point is found, so the worst-case running time of INSERTION-SORT remains $\Theta(n^2)$.

2.3-7

We must decide whether *two* elements of S have a sum equal to x , not whether any single element equals x . We first sort S with MERGE-SORT, and then sweep the sorted array inwards from both ends.

SUM-EXISTS(S, x)

```

1: let  $A[1..n]$  be the elements of  $S$ 
2: MERGE-SORT( $A, 1, n$ )
3:  $low = 1$ 
4:  $high = n$ 
5: while  $low < high$  do
6:    $sum = A[low] + A[high]$ 
7:   if  $sum == x$  then
8:     return TRUE
9:   else if  $sum < x$  then
10:     $low = low + 1$ 
11:   else
12:     $high = high - 1$ 
13: return FALSE

```

The condition $low < high$ guarantees that the two elements summed are distinct positions of A . If $A[low] + A[high] < x$ then no element paired with $A[low]$ can reach x , since $A[high]$ is the largest remaining candidate, and so low may safely be advanced; the symmetric argument justifies decrementing $high$. Every iteration therefore discards an element that cannot participate in any solution, and no pair summing to x is ever lost.

MERGE-SORT takes $\Theta(n \lg n)$ time and the **while** loop of lines 5-13 performs at most n iterations of constant cost, so the total running time is

$$\Theta(n \lg n) + \Theta(n) = \Theta(n \lg n).$$

Problems

2-1

- a. *Proof.* Since INSERTION-SORT has a running time of $\Theta(n^2)$, due to the nested **while** loop of lines 5-7 (p. 26), and the loop of lines 1-7, if we are instead sorting n items divided into n/k sublists, the outer **for** loop will

run k times for each sublist. Since each sublist will contain k items, the **while** loop will run $k - 1$ times for each sublist, so we have a running time of $\Theta(k^2)$ for each sublist, and since there are n/k sublists, the total running time will be

$$\Theta\left(\frac{n}{k} \cdot k^2\right) = \Theta(nk).$$

□

- b.** If we merge the sublists when their length reaches k , we are removing $\lg k$ levels from the recursion tree on p. 38, so we have

$$\lg n - \lg k = \lg n/k$$

levels on the tree. Now, from **a.**, we have the INSERTION-SORT algorithm on the n/k sublists with a running time of $\Theta(nk)$, so the new algorithm will have a running time of

$$\Theta(nk + n \lg(n/k)).$$

Note that the two terms cannot be combined: neither dominates the other for all k , since nk is the larger when k is close to n and $n \lg(n/k)$ is the larger when k is constant.

- c.** We want the largest k for which

$$\Theta(nk + n \lg(n/k)) = \Theta(n \lg n),$$

and expanding $\lg(n/k) = \lg n - \lg k$ gives

$$nk + n \lg n - n \lg k.$$

The term $n \lg n - n \lg k$ never exceeds $n \lg n$, so the sum is $O(n \lg n)$ precisely when $nk = O(n \lg n)$, i.e. when $k = O(\lg n)$. Conversely, if $k = O(\lg n)$ then $\lg k = O(\lg \lg n)$ and

$$nk + n \lg n - n \lg k \geq n \lg n - n \lg \lg n = \Omega(n \lg n),$$

so the running time is $\Theta(n \lg n)$ exactly when $k = \Theta(\lg n)$, and this is the largest such k asymptotically. Any $k = \omega(\lg n)$ makes the nk term $\omega(n \lg n)$, so the modified algorithm would then be asymptotically slower than standard merge sort.

- d.** Asymptotically, part **c.** permits any $k = \Theta(\lg n)$. In practice the constants hidden by the Θ -notation are what matter, so k should be chosen as the largest sublist length at which INSERTION-SORT is still measurably faster than MERGE-SORT on the target machine, which is determined empirically rather than analytically.

2-2

- a. We need to prove a loop invariant for each loop in the algorithm.
- b. *Proof.* At the start of each iteration of the **for** loop of lines 2-4, $A'[1 \dots i]$ consists of elements sorted in ascending order.

Initialisation: For the first loop of lines 2-4, $A'[1 \dots i]$ is a one-element array, $A'[1]$, which is (trivially) sorted.

Maintenance: After each **for** loop, the first out-of-order element from $A.length$ down to $i + 1$ has been swapped with each element before it until it is in the correct position in $A'[1 \dots i]$.

Termination: The **for** loop terminates when $j == i + 1$, so that, at that point, $A'[i] < A'[i + 1]$ and the loop invariant holds for $A'[1 \dots i + 1]$.

□

- c. *Proof.* For each iteration of the **for** loop of lines 1-4, $A'[1 \dots i + 1]$ is sorted in ascending order.

Initialisation: At initialisation, $i = 1$, so $A'[1 \dots i + 1]$ is $A'[1 \dots 2]$ and the termination of **b.** guarantees that this will be sorted.

Maintenance: Each execution of the **for** loop of lines 2-4 sorts $A'[1 \dots i + 1]$, so the loop invariant is maintained.

Termination: The loop terminates when $i == A.length - 1$, so $A'[1 \dots i + 1] = A'[1 \dots n]$ has been sorted.

□

- d. The worst-case running time occurs when A is sorted in descending order, so the **for** loop of lines 1-4 executes $n - 1$ times and the **for** loop of lines 2-4 executes $n - i + 1$ times. Thus we have $(n - 1)(n - i + 1) = \Theta(n^2)$ worst-case running time, equivalent to that of INSERTION-SORT.

2-3

- a. The running time is $\Theta(n)$, since there is only one **for** loop that runs n times.
- b. NAIVE-POLYNOMIAL-EVALUATION($a_0 \dots a_n, n, x$)
 - 1: $y = 0$
 - 2: **for** $i = 0$ **to** n **do**
 - 3: $temp = 1$
 - 4: **for** $j = 1$ **to** i **do**
 - 5: $temp = temp \cdot x$
 - 6: $y = y + a_i \cdot temp$

7: **return** y

The inner **for** loop of lines 4-5 must run exactly i times starting from $temp = 1$, so that $temp = x^i$ when it terminates and the term added in line 6 is $a_i x^i$.

The running time of this algorithm is $\Theta(n^2)$, since the outer **for** loop of lines 2-6 runs $n + 1$ times and for each execution of this loop, the inner **for** loop of lines 4-5 runs i times, where i is the value of the loop iteration variable declared in line 2 of that iteration, giving

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

multiplications in total.

Clearly this algorithm has a much greater running time than Horner's rule.

- c. *Proof. Initialisation:* The initialisation of the **for** loop of lines 2-3 assigns i to n , thus

$$\sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k = \sum_{k=0}^{n-(n+1)} a_{k+n+1} x^k = \sum_{k=0}^{-1} a_{k+n+1} x^k = 0,$$

which is valid, since $y = 0$ upon initialisation of the **for** loop, therefore its initialisation is correct.

Maintenance: For each subsequent iteration of the **for** loop, y is assigned to $a_i + x \cdot y$, so that, for the first **for** loop, y is assigned to $a_1 + x \cdot 0 = a_1$, and

$$\sum_{k=0}^{n-(n-1+1)} a_{k+(n-1)+1} x^k = \sum_{k=0}^0 a_{k+n} x^k = a_1,$$

which is correct.

For subsequent **for** loops, $a_i + x \cdot y$ multiplies the value of the previous **for** loop,

$$\sum_{k=0}^{n-(i+2)} a_{k+i+2} x^k$$

by x , so that we have

$$x \sum_{k=0}^{n-i-2} a_{k+i+2} x^k = \sum_{k=0}^{n-i-2} a_{k+i+2} x^{k+1},$$

and we add a_i to this value, with the result

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i} x^{k+1}.$$

Thus the loop invariant is maintained at each iteration.

Termination: Since the **for** loop terminates at $i = 0$, we have

$$\sum_{k=0}^{n-(i+1)} a_{k+i+1}x^{k-1} = \sum_{k=0}^{n-1} a_{k+1}x^{k-1}$$

at the beginning of the loop, and with one more operation,

$$y = \sum_{k=0}^n a_k x^k,$$

so that the loop terminates correctly. □

- d. Since the loop invariant is initialised, maintained, and terminates correctly, for

$$P(x) = \sum_{k=0}^n a_k x^k,$$

the correct evaluation is performed for $a_0, a_1, \dots, a_n$.

2-4

- a. $(1, 5), (2, 5), (3, 5), (4, 5)$, and $(3, 4)$.
- b. The array containing all elements of the set $\{1, 2, \dots, n\}$ in reverse order has the most possible inversions. Since there are $n - 1$ inversions, starting with 1, $n - 2$ starting with 2, $\dots$, there are

$$\sum_{k=1}^{n-1} k$$

inversions, or

$$\frac{n(n-1)}{2}$$

inversions.

- c. Since the worst-case running time of INSERTION-SORT occurs for exactly the array discussed in **b.** above, and we know that INSERTION-SORT has a worst-case running time which is $\Theta(n^2)$, we see that there is a direct correspondence between the number of inversions in the input array of INSERTION-SORT and its running time. This follows logically from the fact that each element from $j = 2$ to $A.length$ (where A is the input array) must be sorted if there exists an inversion with j as its second coordinate.
- d. As with MERGE-SORT, we have two subalgorithms, one which finds the number of inversions in a given subarray as follows:

```

MERGE-INVERSIONS( $A, p, q, r$ )
1:  $n_1 = q - p + 1$ 
2:  $n_2 = r - q$ 
3: let  $L[1..n_1 + 1]$  and  $R[1..n_2 + 1]$  be new arrays
4: for  $i = 1$  to  $n_1$  do
5:    $L[i] = A[p + i - 1]$ 
6: for  $j = 1$  to  $n_2$  do
7:    $R[j] = A[q + j]$ 
8:  $L[n_1 + 1] = \infty$ 
9:  $R[n_2 + 1] = \infty$ 
10:  $i = 1$ 
11:  $j = 1$ 
12:  $inversions = 0$ 
13:  $counted = \text{FALSE}$ 
14: for  $k = p$  to  $r$  do
15:   if  $counted == \text{FALSE}$  and  $R[j] < L[i]$  then
16:      $inversions = inversions + n_1 - i + 1$ 
17:      $counted = \text{TRUE}$ 
18:   if  $L[i] \leq R[j]$  then
19:      $A[k] = L[i]$ 
20:      $i = i + 1$ 
21:   else
22:      $A[k] = R[j]$ 
23:      $j = j + 1$ 
24:      $counted = \text{FALSE}$ 
25: return  $inversions$ 

```

Now that we have the recursive procedure that separates the array into subarrays and finds the number of inversions in each, we find the total number of inversions using the following algorithms:

```

COUNT-INVERSIONS( $A, p, r$ )
1:  $inversions = 0$ 
2: if  $p < r$  then
3:    $q = \lfloor (p + r) / 2 \rfloor$ 
4:    $inversions = inversions +$ 
     COUNT-INVERSIONS( $A, p, q$ )
5:    $inversions = inversions +$ 
     COUNT-INVERSIONS( $A, q + 1, r$ )
6:    $inversions = inversions +$ 
     MERGE-INVERSIONS( $A, p, q, r$ )
7: return  $inversions$ 

```

The running total must be local to each invocation rather than passed in as a parameter; threading it through the recursive calls and then adding the result to it again would count every inversion of a subarray twice.

Since the algorithm is based entirely upon MERGE-SORT, with almost the

same number of steps, it also has $\Theta(n \lg n)$ running time.

3 Growth of Functions

3.1 Asymptotic notation

3.1-1

Proof. Suppose $\max(f(n), g(n)) = f(n)$, then $f(n) \geq g(n)$ for all $n > n_0$, for some $n_0 \in \mathbb{R}^+$. Then, since the Θ -relation is reflexive, $f(n) = \Theta(f(n))$, so we have some $c_1, c_2, n_0 \in \mathbb{R}^+$ such that $0 \leq c_1 f(n) \leq f(n) \leq c_2 f(n)$ for all $n > n_0$.

Now, since $g(n)$ is asymptotically positive, it follows that $f(n) \leq c_2 f(n) \leq c_2 f(n) + c_2 g(n) = c_2(f(n) + g(n))$. Then, $f(n) = \max(g(n), f(n)) \Rightarrow c_3(f(n) + g(n)) \leq c_1 f(n)$ for some $c_3 \in \mathbb{R}^+$, provided c_3 is sufficiently small and $c_3 < c_1$.

Thus, we have $0 \leq c_3(f(n), g(n)) \leq \max(f(n), g(n)) \leq c_2(f(n) + g(n))$, and we have $\max(f(n), g(n)) = \Theta(f(n) + g(n))$.

A similar argument proves the case where $\max(f(n), g(n)) = g(n)$. $\square$

3.1-2

Proof. Since b is an arbitrary positive real, the binomial theorem gives no finite expansion of $(n + a)^b$, and we instead bound $n + a$ above and below by constant multiples of n .

Take $n_0 = 2|a|$, and let $n \geq n_0$. Then $|a| \leq n/2$, so

$$\frac{n}{2} = n - \frac{n}{2} \leq n - |a| \leq n + a \leq n + |a| \leq n + \frac{n}{2} \leq 2n.$$

Every quantity above is positive, and $t \mapsto t^b$ is monotonically increasing for $b > 0$, so raising the inequality to the power b preserves it:

$$\left(\frac{n}{2}\right)^b \leq (n + a)^b \leq (2n)^b,$$

that is,

$$0 \leq \frac{1}{2^b} n^b \leq (n + a)^b \leq 2^b n^b$$

for all $n \geq n_0$.

Thus with the identifications $c_1 = 2^{-b}$, $c_2 = 2^b$ and $n_0 = 2|a|$, we have $(n + a)^b = \Theta(n^b)$. $\square$

3.1-3

Since big-O notation gives an upper bound on the running time of a function, it does not make sense to say “at least” in reference to an O-class of functions, as “at least” implies that we are referring to a lower bound.

3.1-4

Recall that $g(n) = O(f(n))$ requires only $0 \leq g(n) \leq c \cdot f(n)$ for all $n \geq n_0$, not equality.

Since $2^{n+1} = 2 \cdot 2^n$, taking $c = 2$ and $n_0 = 1$ gives $0 \leq 2^{n+1} \leq c \cdot 2^n$ for all $n \geq n_0$, so we do have $2^{n+1} = O(2^n)$.

However, $2^{2n} = 2^n \cdot 2^n$, so $2^{2n} \leq c \cdot 2^n$ would require $2^n \leq c$ for all $n \geq n_0$. Since 2^n is unbounded, no such constant $c \in \mathbb{R}^+$ exists, and therefore $2^{2n} \neq O(2^n)$.

3.1-5

Proof. ($\Rightarrow$) Suppose $f(n) = \Theta(g(n))$, then we have $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+$, $n, n_0 \in \mathbb{N}$.

But then it follows that $0 \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, so $f(n) = O(g(n))$, and $0 \leq c_1 g(n) \leq f(n)$ for all $n \geq n_0$, so $f(n) = \Omega(g(n))$.

($\Leftarrow$) Now, suppose $f(n) = O(g(n))$, i.e., $0 \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$ and $f(n) = \Omega(g(n))$, i.e., $0 \leq c_1 g(n) \leq f(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+$, $n, n_0 \in \mathbb{N}$. But then we have $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$, so that $f(n) = \Theta(g(n))$. □

3.1-6

Let $M(n)$ denote the worst-case running time of the algorithm on inputs of size n and $m(n)$ its best-case running time, so that for every input of size n with running time $f(n)$ we have $m(n) \leq f(n) \leq M(n)$.

Proof. ($\Rightarrow$) Suppose the running time of the algorithm is $\Theta(g(n))$, i.e., $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$ and every input of size n , where $c_1, c_2 \in \mathbb{R}^+$, $n, n_0 \in \mathbb{N}$.

The worst-case time $M(n)$ is the running time of some particular input of size n , so it satisfies the same bounds; in particular $0 \leq M(n) \leq c_2 g(n)$ for all $n \geq n_0$, and $M(n) = O(g(n))$.

Likewise $m(n)$ is the running time of some particular input of size n , so $0 \leq c_1 g(n) \leq m(n)$ for all $n \geq n_0$, and $m(n) = \Omega(g(n))$.

($\Leftarrow$) Now, suppose the worst-case running time $M(n) = O(g(n))$ and the best-case running time $m(n) = \Omega(g(n))$. Then we have $0 \leq c_1 g(n) \leq m(n)$ and $0 \leq M(n) \leq c_2 g(n)$ for all $n \geq n_0$, where $c_1, c_2 \in \mathbb{R}^+$, $n, n_0 \in \mathbb{N}$. But $m(n) \leq f(n) \leq M(n)$ for every input of size n , so that it follows that

$$0 \leq c_1 g(n) \leq m(n) \leq f(n) \leq M(n) \leq c_2 g(n)$$

for all $n \geq n_0$, i.e., $f(n) = \Theta(g(n))$. □

3.1-7

Proof. Suppose we have some function $f \in \{o(g(n)) \cap \omega(g(n))\}$. Then we have, for any $c_1, c_2 \in \mathbb{R}^+$, that there exists $n_0 > 0$ such that $0 \leq f(n) < c_2 g(n)$ and $0 \leq c_1 g(n) < f(n)$ for all $n \geq n_0$.

But now we have

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

and

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty,$$

so that we have a contradiction. Thus, we conclude that such an f does not exist, and $o(g(n)) \cap \omega(g(n)) = \emptyset$, the empty set. $\square$

3.1-8

$$\begin{aligned} \Omega(g(n, m)) = \{f(n, m) : & \text{there exist positive constants } c, n_0, \text{ and } m_0 \\ & \text{such that } 0 \leq cg(n, m) \leq f(n, m) \\ & \text{for all } n \geq n_0 \text{ or } m \geq m_0\} \end{aligned}$$

and

$$\begin{aligned} \Theta(g(n, m)) = \{f(n, m) : & \text{there exist positive constants } c_1, c_2, n_0, \text{ and } m_0 \\ & \text{such that } 0 \leq c_1 g(n, m) \leq f(n, m) \leq c_2 g(n, m) \\ & \text{for all } n \geq n_0 \text{ or } m \geq m_0\}. \end{aligned}$$

3.2 Standard notations and common functions

3.2-1

Proof. **a.** Suppose $f(n)$ and $g(n)$ are monotonically increasing functions, i.e., $m \leq n \Rightarrow f(m) \leq f(n)$ and $g(m) \leq g(n)$. Then, clearly for $m \leq n$, we have

$$f(m) + g(m) \leq f(n) + g(m) \leq f(n) + g(n),$$

so that $f(n) + g(n)$ is monotonically increasing.

b. Since $m \leq n \Rightarrow g(m) \leq g(n)$ and $f(m) \leq f(n)$, it must also be the case that

$$m \leq n \Rightarrow g(m) \leq g(n) \Rightarrow f(g(m)) \leq f(g(n)),$$

so that $f(g(n))$ is a monotonically increasing function.

c. Suppose further that $f(n)$ and $g(n)$ are nonnegative. Then

$$m \leq n \Rightarrow f(m) \leq f(n) \Rightarrow f(m) \cdot g(n) \leq f(n) \cdot g(n)$$

and

$$m \leq n \Rightarrow g(m) \leq g(n) \Rightarrow f(n) \cdot g(m) \leq f(n) \cdot g(n),$$

so that we have

$$m \leq n \Rightarrow f(m) \cdot g(m) \leq f(n) \cdot g(n),$$

and $f(n) \cdot g(n)$ is a monotonically increasing function. □

3.2-2

Proof. For all $a, b, c > 0 \in \mathbb{R}$, we have

$$\begin{aligned} \log_b c &= \frac{\log_a c}{\log_a b} \\ &= \log_b a \cdot \log_a c \\ &= \log_a c \cdot \log_b a. \end{aligned}$$

Thus we have

$$\begin{aligned} a^{\log_b c} &= a^{\log_a c \cdot \log_b a} \\ &= (a^{\log_a c})^{\log_b a} \\ &= c^{\log_b a}. \end{aligned}$$
□

3.2-3

Proof. Since $n! \leq n^n$ for all $n \geq 1$, we have $\lg(n!) \leq \lg n^n = n \lg n$ for all $n \geq 1$, so that we have $\lg(n!) \leq c_2 \cdot (n \lg n)$ for $n \geq n_0$ with the identifications $c_2 = 1$ and $n_0 = 1$, i.e. $\lg(n!) = O(n \lg n)$.

For the lower bound we cannot take $c_1 = 1$, since $\lg(n!) < n \lg n$ for every $n \geq 2$; we must find some constant $c_1 < 1$ instead. From Stirling's approximation,

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right),$$

and taking logarithms,

$$\begin{aligned} \lg(n!) &= \frac{1}{2} \lg(2\pi n) + n \lg n - n \lg e + \lg\left(1 + \Theta\left(\frac{1}{n}\right)\right) \\ &= n \lg n - n \lg e + \frac{1}{2} \lg n + \frac{1}{2} \lg(2\pi) + \Theta\left(\frac{1}{n}\right). \end{aligned}$$

The terms $\frac{1}{2} \lg n$ and $\frac{1}{2} \lg(2\pi)$ are positive for $n \geq 1$, so discarding them can only weaken the bound, and it suffices to show that $n \lg n - n \lg e \geq \frac{1}{2} n \lg n$.

This holds exactly when

$$\begin{aligned}\frac{1}{2}n \lg n &\geq n \lg e \\ \Leftrightarrow \lg n &\geq 2 \lg e \approx 2.885 \\ \Leftrightarrow n &\geq e^2 \approx 7.39,\end{aligned}$$

so that $\lg(n!) \geq \frac{1}{2}(n \lg n)$ for all $n \geq 8$, and $\lg(n!) = \Omega(n \lg n)$ with $c_1 = \frac{1}{2}$ and $n_0 = 8$.

Since $\lg(n!)$ is both $O(n \lg n)$ and $\Omega(n \lg n)$, exercise 3.1-5 gives $\lg(n!) = \Theta(n \lg n)$.

Now,

$$\frac{n!}{n^n} = \prod_{k=1}^n \frac{k}{n} = \frac{1}{n} \cdot \prod_{k=2}^n \frac{k}{n} \leq \frac{1}{n},$$

since every factor with $k \leq n$ is at most 1. Hence $n!/n^n \rightarrow 0$ as $n \rightarrow \infty$, and $n! = o(n^n)$.

Finally, writing $r_n = n!/2^n$, we have $r_n = r_{n-1} \cdot \frac{n}{2}$, and $\frac{n}{2} \geq 2$ for all $n \geq 4$. Thus $r_n \geq 2r_{n-1}$ from $n = 4$ onwards, so $r_n \rightarrow \infty$ and $n! = \omega(2^n)$. $\square$

3.2-4

Suppose $f(n) = \lceil \lg n \rceil!$ is not polynomially bounded, so that we have $f(n) \neq O(n^k)$ for any constant k , i.e., there do not exist positive constants c and n_0 s.t. $0 \leq n^k \leq c \lceil \lg n \rceil!$ for all $n \geq n_0$, for any k . But we have

$$\begin{aligned}\lceil \lg n \rceil! &< (\lg n + 1)! \\ &= (\lg n + 1)(\lg n)(\lg n - 1) \cdots (2)(1),\end{aligned}$$

and from Stirling's approximation, we have

$$\lceil \lg n \rceil! = \sqrt{2\pi \lceil \lg n \rceil} \left(\frac{\lceil \lg n \rceil}{e} \right)^{\lceil \lg n \rceil} \left(1 + \Theta\left(\frac{1}{\lceil \lg n \rceil} \right) \right)$$

4 Divide-and-Conquer

4.1 The maximum-subarray problem

4.1-1

When every element of A is negative, FIND-MAXIMUM-SUBARRAY returns a single-element subarray $(i, i, A[i])$, where $A[i]$ is the largest, i.e. the least negative, element of A .

To see this, note that appending any further element to a subarray of a wholly negative array strictly decreases its sum. Every subarray of two or more elements therefore has a sum smaller than that of its largest element taken alone, so the maximum is always attained by one of the one-element base cases of the recursion.

4.1-2

BRUTE-FORCE-MAXIMUM-SUBARRAY($A, low, high$)

```
1:  $maxLow = low$ 
2:  $maxHigh = low$ 
3:  $maxSum = A[low]$ 
4: for  $i = low$  to  $high$  do
5:    $sum = 0$ 
6:   for  $j = i$  to  $high$  do
7:      $sum = sum + A[j]$ 
8:     if  $sum > maxSum$  then
9:        $maxSum = sum$ 
10:     $maxLow = i$ 
11:     $maxHigh = j$ 
12: return ( $maxLow, maxHigh, maxSum$ )
```

The running sum is reset at the start of each iteration of the outer **for** loop of lines 4–11, so that sum always holds the value of the subarray $A[i..j]$. Since the outer loop runs n times and the inner **for** loop of lines 6–10 runs at most n times, the running time is $\Theta(n^2)$.

As in exercise 4.1-5, $maxSum$ is initialised to $A[low]$ rather than to 0, so that the algorithm returns the largest element of an array whose entries are all negative.

4.1-3

We have the following output from the program `maxsubarraybench`, which can be found under `cpp/maxsubarraybench/main.cpp`:

```
=====
Results for array of size 19:
=====
Execution time for algorithm "brute_force_find_max_subarray": 0.0005 ms
Execution time for algorithm "find_max_subarray": 0.009958 ms
=====
Results for array of size 20:
=====
Execution time for algorithm "brute_force_find_max_subarray": 0.000417 ms
Execution time for algorithm "find_max_subarray": 0.003625 ms
```

For an array of size $n = 20$, the divide-and-conquer algorithm overtakes the brute-force algorithm. The hybrid approach using brute-force for $n_0 < 20$ seems to run faster than the purely recursive approach for all values of n_0 .

4.1-4

The simplest change is to leave the algorithm untouched and inspect its result. Run `FIND-MAXIMUM-SUBARRAY` as before; if the sum it returns is negative, discard it and return the empty subarray with sum 0 instead, and otherwise return the subarray unchanged.

This is correct because a maximum subarray with a negative sum can only arise when every element of A is negative, as shown in exercise 4.1-1, and in that case the empty subarray, with sum 0, is the better answer under the revised definition. Whenever A contains at least one nonnegative element, the maximum sum is itself nonnegative and the empty subarray cannot improve on it. The extra test costs $\Theta(1)$, so the running time is unchanged.

4.1-5

FIND-MAXIMUM-SUBARRAY($A, low, high$)

```

1:  $maxLow = low$ 
2:  $maxHigh = low$ 
3:  $maxSum = A[low]$ 
4:  $currentLow = low$ 
5:  $currentSum = A[low]$ 
6: for  $j = low + 1$  to  $high$  do
7:   if  $currentSum > 0$  then
8:      $currentSum = currentSum + A[j]$ 
9:   else
10:     $currentLow = j$ 
11:     $currentSum = A[j]$ 
12:   if  $currentSum > maxSum$  then
13:      $maxSum = currentSum$ 
14:      $maxLow = currentLow$ 
15:      $maxHigh = j$ 
16: return ( $maxLow, maxHigh, maxSum$ )

```

The **for** loop of lines 6–16 extends the maximum subarray ending at $j - 1$ by $A[j]$ whenever that running sum is still positive, and otherwise begins a fresh subarray at j . Since each element is examined exactly once, the running time is $\Theta(n)$.

Note that $maxSum$ is initialised to $A[low]$ rather than to the sum of the first two elements, so that a single element is a valid answer and an array of entirely negative values returns its largest element.

4.2 Strassen's algorithm for matrix multiplication

4.2-1

For the matrix product

$$\begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix},$$

we assign

$$A = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}$$

and

$$B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix},$$

so we have

$$\begin{aligned} S_1 &= B_{12} - B_{22} = 8 - 2 = 6, \\ S_2 &= A_{11} + A_{12} = 1 + 3 = 4, \\ S_3 &= A_{21} + A_{22} = 7 + 5 = 12, \\ S_4 &= B_{21} - B_{11} = 4 - 6 = -2, \\ S_5 &= A_{11} + A_{22} = 1 + 5 = 6, \\ S_6 &= B_{11} + B_{22} = 6 + 2 = 8, \\ S_7 &= A_{12} - A_{22} = 3 - 5 = -2, \\ S_8 &= B_{21} + B_{22} = 4 + 2 = 6, \\ S_9 &= A_{11} - A_{21} = 1 - 7 = -6, \\ \text{and } S_{10} &= B_{11} + B_{12} = 6 + 8 = 14. \end{aligned}$$

Then, we recursively multiply the $n/2 \times n/2 = 1 \times 1$ matrices seven times to compute

$$\begin{aligned} P_1 &= A_{11} \cdot S_1 = 1 \cdot 6 = 6, \\ P_2 &= S_2 \cdot B_{22} = 4 \cdot 2 = 8, \\ P_3 &= S_3 \cdot B_{11} = 12 \cdot 6 = 72, \\ P_4 &= A_{22} \cdot S_4 = 5 \cdot (-2) = -10, \\ P_5 &= S_5 \cdot S_6 = 6 \cdot 8 = 48, \\ P_6 &= S_7 \cdot S_8 = -2 \cdot 6 = -12, \\ \text{and } P_7 &= S_9 \cdot S_{10} = -6 \cdot 14 = -84. \end{aligned}$$

Now, we add and subtract the P_i submatrices to obtain

$$\begin{aligned} C_{11} &= P_5 + P_4 - P_2 + P_6 = 48 + (-10) - 8 + (-12) = 18, \\ C_{12} &= P_1 + P_2 = 6 + 8 = 14, \\ C_{21} &= P_3 + P_4 = 72 + (-10) = 62, \\ \text{and } C_{22} &= P_5 + P_1 - P_3 - P_7 = 48 + 6 - 72 - (-84) = 66. \end{aligned}$$

Thus, we have

$$C = A \cdot B = \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}.$$

4.2-2

STRASSEN-MULTIPLY(A, B)

- 1: $n = A.rows$
- 2: let C be a new $n \times n$ matrix


```

3: if  $n == 1$  then
4:    $C_{11} = A_{11} \cdot B_{11}$ 
5:   return  $C$ 
6: else
7:   partition  $A$ ,  $B$ , and  $C$  as in equations (4.9)
8:    $S_1 = B_{12} - B_{22}$ 
9:    $S_2 = A_{11} + A_{12}$ 
10:   $S_3 = A_{21} + A_{22}$ 
11:   $S_4 = B_{21} - B_{11}$ 
12:   $S_5 = A_{11} + A_{22}$ 
13:   $S_6 = B_{11} + B_{22}$ 
14:   $S_7 = A_{12} - A_{22}$ 
15:   $S_8 = B_{21} + B_{22}$ 
16:   $S_9 = A_{11} - A_{21}$ 
17:   $S_{10} = B_{11} + B_{12}$ 
18:   $P_1 = \text{STRASSEN-MULTIPLY}(A_{11}, S_1)$ 
19:   $P_2 = \text{STRASSEN-MULTIPLY}(S_2, B_{22})$ 
20:   $P_3 = \text{STRASSEN-MULTIPLY}(S_3, B_{11})$ 
21:   $P_4 = \text{STRASSEN-MULTIPLY}(A_{22}, S_4)$ 
22:   $P_5 = \text{STRASSEN-MULTIPLY}(S_5, S_6)$ 
23:   $P_6 = \text{STRASSEN-MULTIPLY}(S_7, S_8)$ 
24:   $P_7 = \text{STRASSEN-MULTIPLY}(S_9, S_{10})$ 
25:   $C_{11} = P_5 + P_4 - P_2 + P_6$ 
26:   $C_{12} = P_1 + P_2$ 
27:   $C_{21} = P_3 + P_4$ 
28:   $C_{22} = P_5 + P_1 - P_3 - P_7$ 
29:  return  $C$ 

```

4.2-3

We pad the operands to a power of two, which the partitioning of equations (4.9) accepts, and discard the padding afterwards.

Let $m = 2^{\lceil \lg n \rceil}$ be the smallest power of 2 with $m \geq n$.

STRASSEN-MULTIPLY-PADDED(A, B)

```

1:  $n = A.\text{rows}$ 
2:  $m = 2^{\lceil \lg n \rceil}$ 
3: let  $A'$  and  $B'$  be new  $m \times m$  matrices with every entry 0
4: for  $i = 1$  to  $n$  do
5:   for  $j = 1$  to  $n$  do
6:      $a'_{ij} = a_{ij}$ 
7:      $b'_{ij} = b_{ij}$ 
8:  $C' = \text{STRASSEN-MULTIPLY}(A', B')$ 
9: return the leading  $n \times n$  submatrix of  $C'$ 

```

The padding is correct because the added rows and columns are entirely

zero. For $i, j \leq n$ we have

$$c'_{ij} = \sum_{k=1}^m a'_{ik} b'_{kj} = \sum_{k=1}^n a_{ik} b_{kj} + \sum_{k=n+1}^m 0 \cdot 0 = c_{ij},$$

so that the leading $n \times n$ submatrix of C' is exactly AB .

For the running time, we note since m is the smallest power of 2 at least n , we have $m < 2n$. Copying the entries costs $\Theta(m^2)$, and STRASSEN-MULTIPLY on an $m \times m$ matrix costs $\Theta(m^{\lg 7})$, which dominates the running time. Thus the total is

$$\Theta(m^{\lg 7}) = O((2n)^{\lg 7}) = O(2^{\lg 7} \cdot n^{\lg 7}) = O(n^{\lg 7}),$$

since $2^{\lg 7} = 7$ is constant, and the product is trivially $\Omega(n^{\lg 7})$, so that the algorithm runs in $\Theta(n^{\lg 7})$ time.

A Summations

A.1 Summation formulas and properties

A.1-1

$$\begin{aligned} \sum_{k=1}^n (2k-1) &= \sum_{k=1}^n 2k - \sum_{k=1}^n 1 \\ &= 2 \sum_{k=1}^n k - \sum_{k=1}^n 1 \\ &= n(n+1) - n \\ &= n^2. \end{aligned}$$

A.1-2

Proof. We have

$$\begin{aligned} \sum_{k=1}^n \frac{1}{2k-1} &= \sum_{k=1}^n \frac{1}{2} \cdot \frac{1}{k - \frac{1}{2}} \\ &= \frac{1}{2} \sum_{k=1}^n \frac{1}{k - \frac{1}{2}} \\ &= \frac{1}{2} \left[\ln \left(n - \frac{1}{2} \right) + O(1) \right] \\ &= \frac{1}{2} \ln \left(n - \frac{1}{2} \right) + O(1) \\ &= \frac{1}{2} \ln(n) + O(1) \\ &= \ln \sqrt{n} + O(1). \end{aligned}$$

□

A.1-3*Proof.* We have

$$\begin{aligned}
\frac{d}{dx} \sum_{k=0}^{\infty} kx^k &= \sum_{k=1}^{\infty} k^2 x^{k-1} \\
&= \frac{d}{dx} \frac{x}{(1-x)^2} \\
&= \frac{(1-x)^2 + 2x(1-x)}{(1-x)^4} \\
&= \frac{(1-x) + 2x}{(1-x)^3} \\
&= \frac{x+1}{(1-x)^3}.
\end{aligned}$$

Thus we have

$$\begin{aligned}
x \sum_{k=0}^{\infty} k^2 x^{k-1} &= \sum_{k=0}^{\infty} k^2 x^k \\
&= \frac{x(x+1)}{(1-x)^3},
\end{aligned}$$

for $|x| < 1$.

□

A.1-4*Proof.* We have

$$\begin{aligned}
\frac{k-1}{2^k} &= \frac{k}{2^k} - \frac{1}{2^k} \\
&= kx^k - x^k,
\end{aligned}$$

for $x = \frac{1}{2}$. Now, since $|x| < 1$, from **A.6** and **A.8**, we have

$$\begin{aligned}
\sum_{k=0}^{\infty} (k-1)/2^k &= \sum_{k=0}^{\infty} kx^k - \sum_{k=0}^{\infty} x^k \\
&= \frac{x}{(1-x)^2} - \frac{1}{1-x} \\
&= \frac{x - (1-x)}{(1-x)^2} \\
&= \frac{2x-1}{(1-x)^2} \\
&= \frac{1-1}{(\frac{1}{2})^2} \\
&= 0.
\end{aligned}$$

□

A.1-5

Proof. For $|x| < 1$ we first evaluate the related series in closed form by treating it as a geometric series in x^2 :

$$\sum_{k=0}^{\infty} x^{2k+1} = x \sum_{k=0}^{\infty} (x^2)^k = \frac{x}{1-x^2}.$$

Since a power series may be differentiated term by term inside its interval of convergence, and $\frac{d}{dx} x^{2k+1} = (2k+1)x^{2k}$, we have

$$\begin{aligned}
\sum_{k=0}^{\infty} (2k+1)x^{2k} &= \frac{d}{dx} \sum_{k=0}^{\infty} x^{2k+1} \\
&= \frac{d}{dx} \frac{x}{1-x^2} \\
&= \frac{(1-x^2) - x \cdot (-2x)}{(1-x^2)^2} \\
&= \frac{1-x^2+2x^2}{(1-x^2)^2} \\
&= \frac{1+x^2}{(1-x^2)^2}.
\end{aligned}$$

□

A.1-6

Proof. Here n is a constant, independent of i , so the sum has a fixed number of terms. For each $k = 1, 2, \dots, n$, let $g_k(i) \in O(f_k(i))$ be an arbitrary member of

that set, so that by definition there are positive constants c_k and i_k with

$$0 \leq g_k(i) \leq c_k f_k(i) \quad \text{for all } i \geq i_k.$$

Put $c = \max\{c_1, c_2, \dots, c_n\}$ and $i_0 = \max\{i_1, i_2, \dots, i_n\}$, both of which exist and are finite precisely because there are only n terms. Then for every $i \geq i_0$, all n bounds hold simultaneously and

$$0 \leq \sum_{k=1}^n g_k(i) \leq \sum_{k=1}^n c_k f_k(i) \leq c \sum_{k=1}^n f_k(i).$$

Since $\sum_{k=1}^n g_k(i)$ is an arbitrary member of $\sum_{k=1}^n O(f_k(i))$, and we have exhibited constants c and i_0 witnessing that it lies in $O(\sum_{k=1}^n f_k(i))$, it follows that

$$\sum_{k=1}^n O(f_k(i)) = O\left(\sum_{k=1}^n f_k(i)\right).$$

Note that the finiteness of n is essential: the argument fails if n is permitted to grow with i , since $\max_k c_k$ need not then be bounded. $\square$